

Tài liệu hướng dẫn thực hành với Hadoop, Kafka, Spark

07/2025

Mục lục

Phần 1: Giới thiệu.....	1
1.1 Mục đích và ý nghĩa tài liệu.....	1
1.2 Yêu cầu chuẩn bị.....	1
Phần 2: Khởi tạo VM.....	2
Phần 3: L03 Spark WordCount.....	2

Phần 1: Giới thiệu

1.1 Mục đích và ý nghĩa tài liệu

Tài liệu này hướng dẫn người dùng cài đặt Hadoop, Yarn, Kafka, Spark, Nginx, Fluentbit lên VM của GCP, và thực hiện một số bài thực hành.

1.2 Yêu cầu chuẩn bị

- Một máy tính (Laptop, PC) có kết nối internet.
- Một tài khoản Google Cloud
- Ứng dụng VSCode

Phần 2: Khởi tạo VM

Tham khảo từ bài L01 tới trang 13, khởi tạo một VM với các thông số như sau:

E2, standard, 8vCPU, 32GB Ram, 50GB Disk, Ubuntu 24.04 LTS (amd).

Sau khi khởi tạo thành công, tiến hành thiết lập quyền truy cập vào root user, và truy cập VM bằng VSCode

Phần 3: L03 Spark WordCount

Mô tả: Trong phần này, chúng ta sẽ dựng cụm Hadoop để lưu các file text, sau đó dùng Spark để thực hiện xử lý đếm số lượng từ. Các chương trình của Spark có thể chạy trên nhiều loại cụm khác nhau như: cụm Spark, cụm Yarn, cụm K8S, Mesos. Trong bài này chúng ta sẽ sử dụng cụm Yarn để chạy cả Spark và MapReduce.

Bước 1: Cài đặt Docker:

```
cd /root
apt update
snap install docker
```

Bước 2: Clone code

```
git clone https://github.com/BKCLab/bigdata-learning.git
```

Dùng VSCode mở thư mục bigdata-learning/l3

Bước 3: Tạo network cho bài thực hành

```
docker network create bigdata-network
```

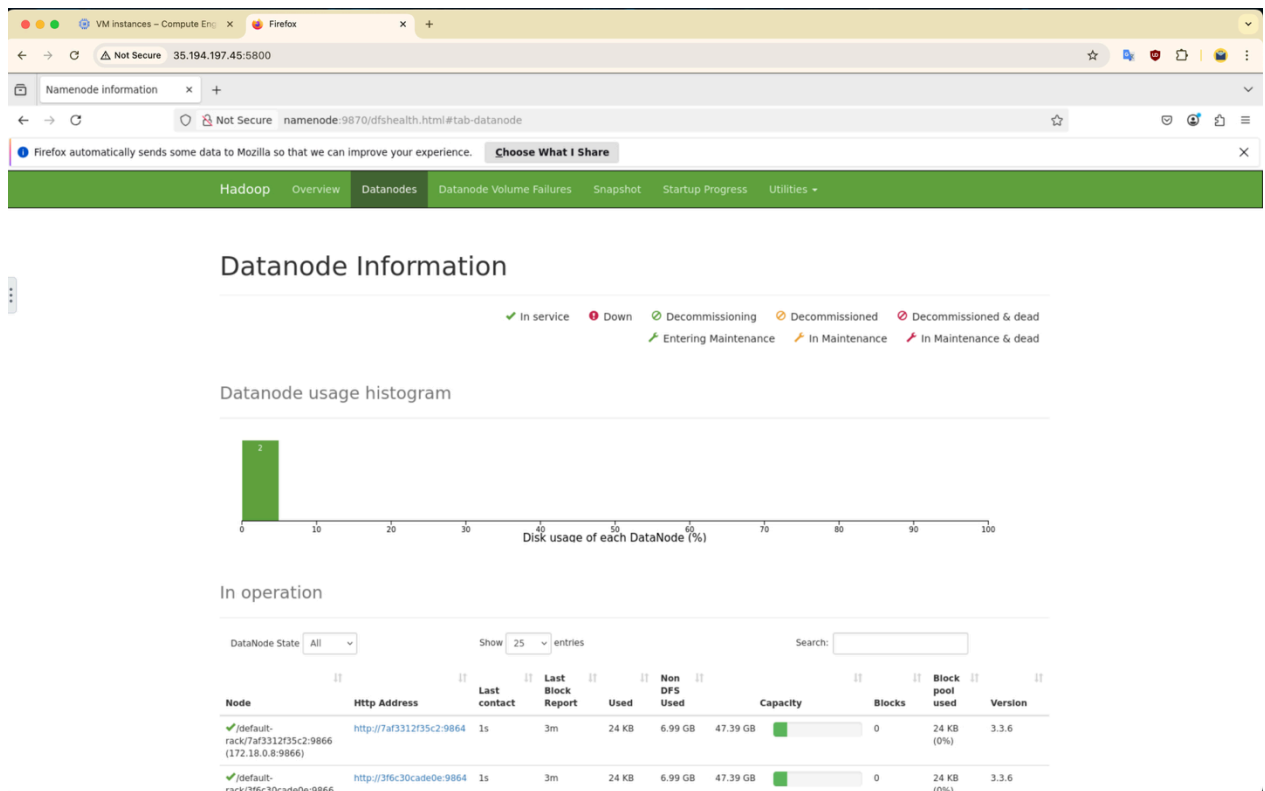
Bước 4: Chạy cụm Hadoop và Yarn

```
cd hadoop
docker compose up
```

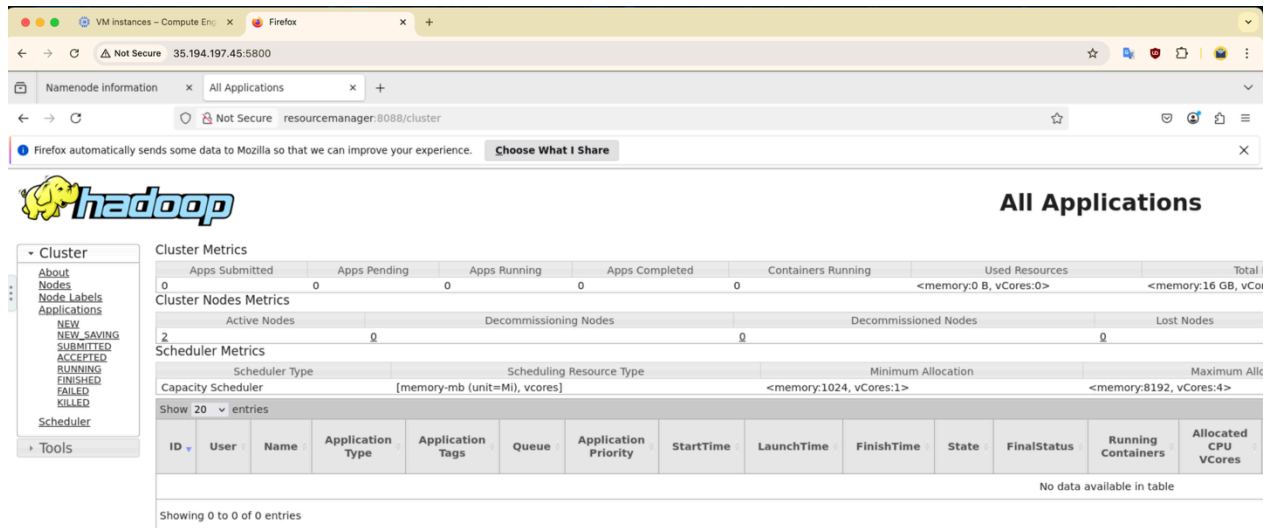
Câu lệnh này sẽ tạo dựng một cụm hadoop gồm 1 namenode + 2 datanode, 1 yarn resource-manager và 2 node-manager, đồng thời compose cũng bao gồm 1 trình duyệt firefox – được sử dụng để dễ dàng truy cập tới các services trong cùng mạng.

Sau khi câu lệnh thực thi xong, truy cập địa chỉ: <http://<vm-ip>:5800> để mở firefox

Để truy cập vào UI của HDFS: gõ <http://namenode:9870> (có thể bạn cần phải tự gõ, vì firefox chạy trong container không hỗ trợ copy paste)



Truy cập Yarn tại: <http://resourcemanager:8088>



Bước 5: Tạo file dữ liệu:

Di chuyển tới thư mục l3/hadoop/gen-data
Mở một terminal mới và download Nodejs:

```
curl -o-  
https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.3/install.sh |  
bash  
source ~/.bashrc  
nvm install 20  
node -v
```

Chạy lệnh sinh file dữ liệu:

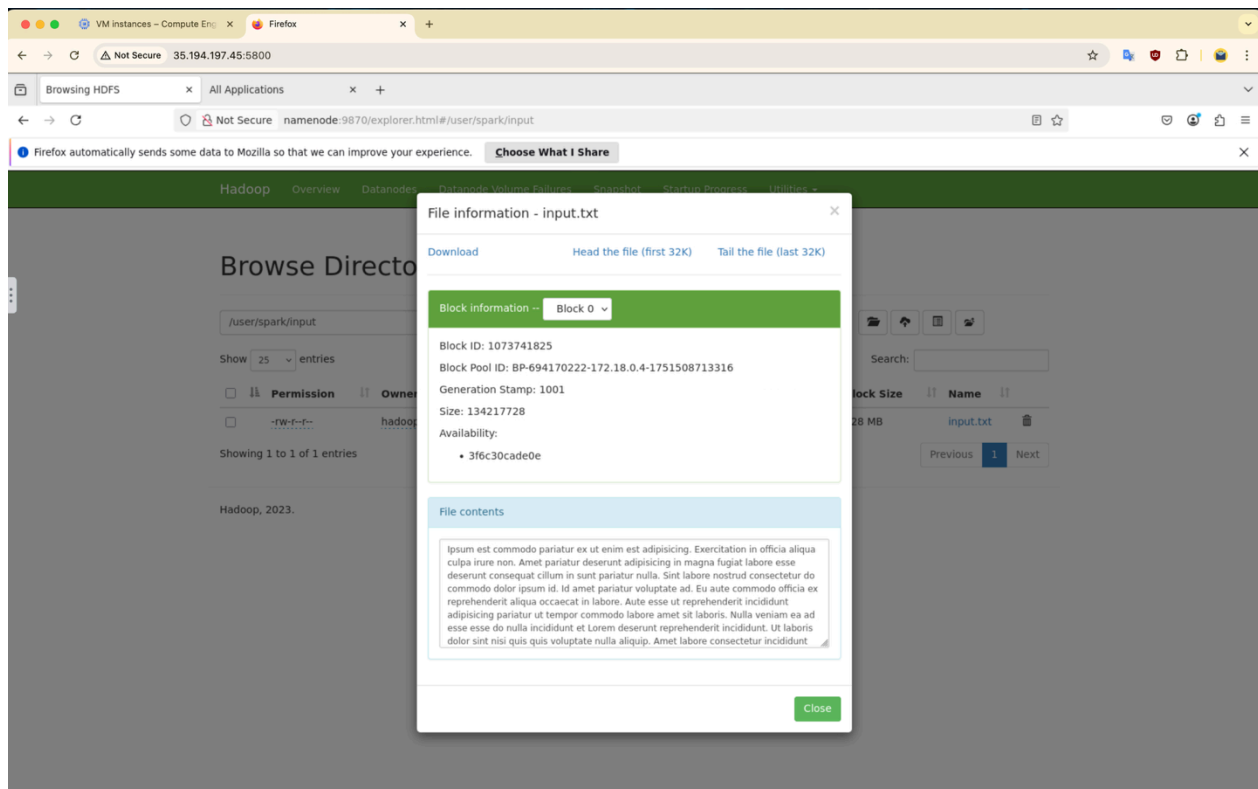
```
npm i  
node main 1024 input.txt
```

Trong đó:
1024 là kích thước file tính theo MB
input.txt là tên file

Thực hiện các lệnh sau để đưa dữ liệu vào hdfs:

```
docker compose exec -it namenode /bin/bash  
hdfs dfs -mkdir -p /user/spark/input  
hdfs dfs -put /opt/gen-data/input.txt /user/spark/input/  
hdfs dfs -chmod -R 777 /
```

Bạn có thể kiểm tra kết quả và xem nội dung file bằng Hadoop UI:



Bước 6: Chạy Spark wordcount

Mở một terminal mới tại l3/hadoop:

```
docker compose exec spark-client spark-submit \
  --master yarn \
  --deploy-mode client \
  --conf spark.yarn.resource.manager.hostname=resourcemanager \
  --num-executors 4 \
  --executor-cores 1 \
  --executor-memory 2g \
  /opt/spark-apps/simple_spark_app.py \
  /user/spark/input/input.txt
```

Trong khi chương trình đang chạy, bạn có thể theo dõi tiến trình bằng Yarn UI hoặc terminal:

Firefox browser window showing the SimpleSparkYARNApp application UI. The address bar shows the URL: resourceManager:8088/proxy/application_1751508718064_0002/executors/. The page title is "SimpleSparkYARNApp application UI". The navigation bar includes links for Jobs, Stages, Storage, Environment, Executors, and SQL / DataFrame. The Executors tab is selected.

Executors

Show Additional Metrics

Summary

	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Excluded
Active(5)	0	264.2 KIB / 4 GIB	0.0 B	4	6	0	0	6	44 s (99.0 ms)	0.0 B	0.0 B	0.0 B	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0
Total(5)	0	264.2 KIB / 4 GIB	0.0 B	4	6	0	0	6	44 s (99.0 ms)	0.0 B	0.0 B	0.0 B	0

Executors

Show 20 entries

Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Thread Dump	Heap Histogram	Add Time	Re Ti
driver	spark-client:33357	Active	0	52.8 KIB / 434.4 MIB	0.0 B	0	0	0	0	0	44 s (99.0 ms)	0.0 B	0.0 B	0.0 B		Thread Dump	Heap Histogram	2025-07-03 02:29:09	-
1	572be712b72a:36363	Active	0	52.8 KIB / 912.3 MIB	0.0 B	1	2	0	0	2	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	stdout stderr	Thread Dump	Heap Histogram	2025-07-03 02:29:20	-
2	03ad064e000e:44579	Active	0	52.8 KIB / 912.3 MIB	0.0 B	1	1	0	0	1	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	stdout stderr	Thread Dump	Heap Histogram	2025-07-03 02:29:13	-

Terminal window showing the output of the Spark application. The output includes logs from the DAGScheduler, MemoryStore, BlockManager, and YarnScheduler. The logs show the execution of a MapReduce job, including the submission of tasks, the execution of tasks, and the completion of the job. The output also includes a word count table.

```
25/07/03 02:29:37 INFO DAGScheduler: Parents of final stage: List(ShuffleMapStage 1)
25/07/03 02:29:37 INFO DAGScheduler: Missing parents: List()
25/07/03 02:29:37 INFO DAGScheduler: Submitting 1 missing tasks from ResultStage 2 (MapPartitionsRDD[6] at showString at NativeMethodAccessorImpl.java:0), which has no missing parents
25/07/03 02:29:37 INFO MemoryStore: Block broadcast_2 stored as values in memory (estimated size 48.0 KIB, free 434.1 MIB)
25/07/03 02:29:37 INFO MemoryStore: Block broadcast_2_piece0 stored as bytes in memory (estimated size 18.8 KIB, free 434.1 MIB)
25/07/03 02:29:37 INFO BlockManagerInfo: Added broadcast_2_piece0 in memory on 03ad064e000e:44579 (size: 18.8 KIB, free: 434.3 MIB)
25/07/03 02:29:37 INFO SparkContext: Created broadcast 2 from broadcast at DAGScheduler.scala:1508
25/07/03 02:29:37 INFO DAGScheduler: Submitting 1 missing tasks from ResultStage 2 (MapPartitionsRDD[6] at showString at NativeMethodAccessorImpl.java:0) (first 15 tasks are for partitions Vector(0))
25/07/03 02:29:37 INFO YarnScheduler: Adding task set 2.0 with 1 tasks resource profile 0
25/07/03 02:29:37 INFO TaskSetManager: Starting task 0.0 in stage 2.0 (TID 9) (03ad064e000e, executor 2, partition 0, NODE_LOCAL, 7626 bytes)
25/07/03 02:29:37 INFO BlockManagerInfo: Added broadcast_2_piece0 in memory on 03ad064e000e:44579 (size: 18.8 KIB, free: 912.2 MIB)
25/07/03 02:29:37 INFO BlockManagerInfo: Removed broadcast_1_piece0 on 572be712b72a:36363 in memory (size: 17.9 KIB, free: 912.3 MIB)
25/07/03 02:29:37 INFO BlockManagerInfo: Removed broadcast_1_piece0 on 572be712b72a:37381 in memory (size: 17.9 KIB, free: 912.3 MIB)
25/07/03 02:29:37 INFO BlockManagerInfo: Removed broadcast_1_piece0 on 03ad064e000e:44579 in memory (size: 17.9 KIB, free: 912.2 MIB)
25/07/03 02:29:37 INFO BlockManagerInfo: Removed broadcast_1_piece0 on 03ad064e000e:36803 in memory (size: 17.9 KIB, free: 912.3 MIB)
25/07/03 02:29:38 INFO TaskSetManager: Finished task 0.0 in stage 2.0 (TID 9) in 661 ms on 03ad064e000e (executor 2) (1/1)
25/07/03 02:29:38 INFO YarnScheduler: Removed TaskSet 2.0, whose tasks have all completed, from pool
25/07/03 02:29:38 INFO DAGScheduler: ResultStage 2 (showString at NativeMethodAccessorImpl.java:0) finished in 0.690 s
25/07/03 02:29:38 INFO DAGScheduler: Job 1 is finished. Cancelling potential speculative or zombie tasks for this job
25/07/03 02:29:38 INFO YarnScheduler: Killing all running tasks in stage 2: Stage finished
25/07/03 02:29:38 INFO CodeGenerator: Code generated in 14.65824 ms
```

word	count
Sixt	258046
Ellis	258187
consectetur	257478
Official	258089
veniam	2863461
Eui	257810
non	258257
ipsum	257710
tempor	257388
Sunt	258064
Adipiscing	257443
cillum	2863168
duis	257931
velit	2861768
Laborum	286214
fugiat	2864528
occaecat	258189
sunt	257708
exercitation	258248
Aute	257729

```
only showing top 20 rows

25/07/03 02:29:39 INFO SparkContext: SparkContext is stopping with exitCode 0.
25/07/03 02:29:39 INFO SparkUI: Stopped Spark web UI at http://spark-client:4040
25/07/03 02:29:39 INFO YarnClientSchedulerBackend: Interrupting monitor thread
25/07/03 02:29:39 INFO YarnClientSchedulerBackend: Shutting down all executors
25/07/03 02:29:39 INFO YarnClientSchedulerBackend: Asking each executor to shut down
25/07/03 02:29:39 INFO YarnClientSchedulerBackend: YARN client scheduler backend Stopped
25/07/03 02:29:39 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
25/07/03 02:29:39 INFO MemoryStore: MemoryStore cleared
25/07/03 02:29:39 INFO BlockManager: BlockManager stopped
25/07/03 02:29:39 INFO BlockManagerMaster: BlockManagerMaster stopped
25/07/03 02:29:39 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
25/07/03 02:29:39 INFO SparkContext: Successfully stopped SparkContext
Spark application finished.
25/07/03 02:29:40 INFO ShutdownHookManager: Shutdown hook called
25/07/03 02:29:40 INFO ShutdownHookManager: Deleting directory /tmp/spark-2b11fe5-f560-4583-a38e-57d7aab1d15/pyspark-c445c3d3-7e81-4f54-a3eb-b3c2e227d774
25/07/03 02:29:40 INFO ShutdownHookManager: Deleting directory /tmp/spark-8eac8b81-0613-4b5c-b380-98eac8b15801
25/07/03 02:29:40 INFO ShutdownHookManager: Deleting directory /tmp/spark-2b11fe5-f560-4583-a38e-57d7aab1d15
root@bigdata-0384:~/bigdata-learning/13/hadoop#
```

Bước 7: Thực hiện so sánh giữa MapReduce và Spark:

Thực hiện sinh các file dữ liệu với kích thước khác nhau, đưa các file này vào hdfs, rồi thực hiện chạy WordCount bằng MapReduce và Spark rồi so sánh thời gian thực thi.

(Bạn cũng có thể thử điều chỉnh các tham số của Spark như số lượng executor, tài nguyên cho mỗi executor, tham số shuffle.partitions v.v khi chạy Spark để so sánh kết quả)

Câu lệnh tham khảo để chạy Yarn WordCount:

```
docker compose exec -it namenode /bin/bash
```

```
yarn jar
```

```
$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.j
```

```
ar wordcount /user/spark/input /user/spark/output
```